

Fiche récapitulative – STA211

Réseaux de neurones – Cours 1

Du neurone formel au perceptron multicouche, rétropropagation et optimisation

Nicolas Thome (CNAM)

14 juin 2026

Résumé

Cette fiche présente les fondements des réseaux de neurones profonds (Deep Learning), couvrant le neurone formel (McCulloch & Pitts, 1943), le perceptron, le perceptron multicouche (MLP), l'apprentissage par rétropropagation du gradient (backpropagation), les fonctions d'activation (seuil, sigmoïde, tanh, softmax), et les aspects pratiques d'optimisation (descente de gradient stochastique, régularisation).

Table des matières

1	Contexte	2
2	Le neurone formel	2
2.1	Définition (McCulloch & Pitts, 1943)	2
2.2	Fonctions d'activation	2
2.3	Classification binaire	3
3	Le perceptron	3
3.1	Classification multiclasse	3
3.2	Softmax	3
3.3	Limite de la classification linéaire	3
4	Perceptron multicouche (MLP)	3
4.1	Architecture	3
4.2	Réseaux profonds (DNN)	3
5	Apprentissage	3
5.1	Apprentissage supervisé	3
5.2	Fonction de perte : entropie croisée	4
5.3	Descente de gradient	4
5.4	Rétropropagation (backpropagation)	4
5.5	Descente de gradient stochastique (SGD)	4
5.6	Plan de taux d'apprentissage (learning rate decay)	5
6	Régularisation et généralisation	5
6.1	Sur-apprentissage (overfitting)	5
6.2	Régularisation	5
6.3	Hyper-paramètres	5
	Questions clés (QCM)	5

Contexte

Le Deep Learning constitue une rupture dans la reconnaissance des signaux de bas niveau (images, audio, texte). Avant le DL, les représentations intermédiaires étaient conçues artisanalement (hand-crafted features). Depuis le DL, ces représentations sont apprises automatiquement, ce qui offre :

- Des performances expérimentales supérieures.
- Un apprentissage de représentations de plus haut niveau d'abstraction.
- Une méthodologie commune et indépendante du domaine.

Le neurone formel

Définition (McCulloch & Pitts, 1943)

Le neurone formel est l'unité de base des réseaux de neurones. Il réalise une transformation d'une entrée vectorielle $\mathbf{x} \in \mathbb{R}^m$ en une sortie scalaire $\hat{y} \in \mathbb{R}$ en deux étapes :

1. **Combinaison linéaire (affine)** : $s = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^m w_i x_i + b$
2. **Activation non linéaire** : $\hat{y} = f(s)$

où $\mathbf{w} \in \mathbb{R}^m$ est le vecteur de poids synaptiques, $b \in \mathbb{R}$ le biais, et f la fonction d'activation.

Fonctions d'activation

Fonction seuil (Heaviside) :

$$H(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{sinon} \end{cases} \quad (1)$$

Interprétation biologique : le neurone est activé ($\hat{y} = 1$) si la somme pondérée dépasse le seuil $-b$.

Fonction sigmoïde :

$$\sigma(z) = \frac{1}{1 + e^{-az}} \quad (2)$$

Propriétés : lisse, dérivable, interprétation probabiliste ($\hat{y} \sim P(1|\mathbf{x})$). Saturante aux extrémités.

Fonction tanh :

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (3)$$

Classification binaire

Avec une activation sigmoïde, la sortie du neurone $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ représente la probabilité d'appartenance à la classe 1 :

$$\mathbf{x} \text{ est classe 1 si } \mathbf{w}^\top \mathbf{x} + b > 0 \quad (\hat{y} > 0,5) \quad (4)$$

La frontière de décision est **linéaire** : $\mathbf{w}^\top \mathbf{x} + b = 0$.

Le perceptron

Classification multiclasse

Pour traiter K classes, on utilise K neurones en sortie (perceptron). Chaque neurone de sortie k calcule :

$$s_k = \mathbf{w}_k^\top \mathbf{x} + b_k, \quad \hat{y}_k = f(s_k) \quad (5)$$

En écriture matricielle : $\mathbf{s} = \mathbf{x}\mathbf{W} + \mathbf{b}$, où $\mathbf{W} \in \mathbb{R}^{m \times K}$ et $\mathbf{b} \in \mathbb{R}^{1 \times K}$.

Softmax

Pour la classification multiclasse, on utilise l'activation **softmax** :

$$\hat{y}_k = \frac{e^{s_k}}{\sum_{k'=1}^K e^{s_{k'}}} \quad (6)$$

Interprétation probabiliste : $\hat{y}_k \sim P(k|\mathbf{x}, \mathbf{W})$. Le perceptron avec softmax équivaut à la **régression logistique** (LR).

Limite de la classification linéaire

Le perceptron monocouche est limité aux frontières de décision linéaires. Le problème du XOR (ou exclusif) n'est pas séparable linéairement.

Perceptron multicouche (MLP)

Architecture

On ajoute une ou plusieurs **couches cachées** entre l'entrée et la sortie :

$$\mathbf{h} = f(\mathbf{x}\mathbf{W}_h + \mathbf{b}_h), \quad \hat{\mathbf{y}} = f(\mathbf{h}\mathbf{W}_y + \mathbf{b}_y) \quad (7)$$

$\mathbf{h} \in \mathbb{R}^L$ est la couche cachée (projection de $\mathbf{x}$ dans un nouvel espace). Avec une activation non linéaire f , la frontière de décision devient **non linéaire**.

Réseaux profonds (DNN)

En ajoutant davantage de couches cachées, on obtient un réseau de neurones profond (Deep Neural Network). Chaque couche projette la couche précédente dans un nouvel espace, apprenant progressivement des représentations intermédiaires utiles pour la tâche.

Apprentissage

Apprentissage supervisé

Soit un ensemble d'apprentissage $\mathcal{A} = \{(\mathbf{x}_i, \mathbf{y}_i^*)\}_{i=1}^N$, avec $\mathbf{y}_i^*$ encodé en one-hot. On définit une fonction de perte $\ell(\hat{\mathbf{y}}_i, \mathbf{y}_i^*)$ et on cherche $\mathbf{W} = \arg \min \mathcal{L}$.

Fonction de perte : entropie croisée

Pour la classification, on utilise l'entropie croisée (Cross-Entropy) :

$$\ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*) = - \sum_{c=1}^K y_{c,i}^* \log \hat{y}_{c,i} = - \log \hat{y}_{c^*,i} \quad (8)$$

La perte totale sur l'ensemble d'apprentissage :

$$\mathcal{L}_{CE}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{i=1}^N \ell_{CE}(\hat{\mathbf{y}}_i, \mathbf{y}_i^*) \quad (9)$$

Descente de gradient

On met à jour les paramètres dans la direction opposée au gradient :

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \frac{\partial \mathcal{L}_{CE}}{\partial \mathbf{W}} \quad (10)$$

où η est le taux d'apprentissage (learning rate).

Rétropropagation (backpropagation)

Le gradient se calcule par **rétropropagation** de l'erreur en utilisant la règle de dérivation en chaîne :

$$\frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial x} \quad (11)$$

Pour la régression logistique :

$$\frac{\partial \ell_{CE}}{\partial \hat{y}_i} = - \frac{1}{\hat{y}_{c^*,i}} \quad (12)$$

$$\frac{\partial \ell_{CE}}{\partial \mathbf{s}_i} = \hat{\mathbf{y}}_i - \mathbf{y}_i^* = \boldsymbol{\delta}_i^y \quad (13)$$

$$\frac{\partial \ell_{CE}}{\partial \mathbf{W}} = \mathbf{x}_i^\top \boldsymbol{\delta}_i^y \quad (14)$$

Pour le perceptron (1 couche cachée) :

$$\boldsymbol{\delta}_i^h = \boldsymbol{\delta}_i^y \mathbf{W}_y^\top \odot \mathbf{h}_i \odot (1 - \mathbf{h}_i) \quad (15)$$

$$\frac{\partial \ell}{\partial \mathbf{W}_h} = \mathbf{x}_i^\top \boldsymbol{\delta}_i^h \quad (16)$$

$$\frac{\partial \ell}{\partial \mathbf{W}_y} = \mathbf{h}_i^\top \boldsymbol{\delta}_i^y \quad (17)$$

Cas général (couche l) :

$$\frac{\partial \mathcal{L}}{\partial \mathbf{U}_l} = \boldsymbol{\Delta}_l = \boldsymbol{\Delta}_{l+1} \mathbf{W}_{l+1}^\top \odot \mathbf{H}_l \odot (1 - \mathbf{H}_l) \quad (18)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_l} = \mathbf{H}_{l-1}^\top \boldsymbol{\Delta}_l \quad (19)$$

Descente de gradient stochastique (SGD)

Le calcul du gradient complet $\nabla_{\mathbf{w}}$ sur tout l'ensemble d'apprentissage est trop coûteux. Approximations :

- **En ligne (online)** : un seul exemple par mise à jour.
- **Minibatch** : $B < N$ exemples par mise à jour.

Le gradient bruité permet plus de mises à jour et une convergence plus rapide en pratique.

Plan de taux d'apprentissage (learning rate decay)

Le taux d'apprentissage η peut décroître au cours de l'entraînement :

- Décroissance inverse : $\eta_t = \frac{\eta_0}{1+r \cdot t}$
- Décroissance exponentielle : $\eta_t = \eta_0 e^{-\lambda t}$
- Décroissance par paliers : $\eta_t = \eta_0 r^{\lfloor t/t_u \rfloor}$

Régularisation et généralisation

Sur-apprentissage (overfitting)

L'objectif n'est pas de minimiser la perte sur l'ensemble d'apprentissage, mais de généraliser à des données non vues. Un écart élevé entre la perte d'apprentissage et la perte de test indique du sur-apprentissage.

Régularisation

Ajout d'un terme de pénalité $R(\mathbf{W})$ dans l'objectif :

$$\mathcal{L}(\mathbf{W}) = \mathcal{L}_{CE}(\mathbf{W}) + \alpha R(\mathbf{W}) \quad (20)$$

- L_2 (**weight decay**) : $R(\mathbf{W}) = \|\mathbf{W}\|^2$ (le plus courant).
- L_1 : $R(\mathbf{W}) = \|\mathbf{W}\|_1$ (favorise la parcimonie).
- **Dropout** : désactive aléatoirement des neurones pendant l'entraînement.

Hyper-paramètres

Réseaux de neurones : nombreux hyper-paramètres à régler :

- Paramètres d'optimisation : learning rate, weight decay, plan de decay, nombre d'époques.
- Paramètres d'architecture : nombre de couches, nombre de neurones, type de non-linéarité.

L'ajustement se fait sur un ensemble de validation.

Questions clés (QCM)

1. Quelle est la sortie d'un neurone formel ?

- (a) $\hat{y} = \mathbf{w}^T \mathbf{x} + b$
- (b) $\hat{y} = f(\mathbf{w}^T \mathbf{x} + b)$
- (c) $\hat{y} = \mathbf{x} \mathbf{W} + \mathbf{b}$
- (d) $\hat{y} = f(\mathbf{x})$

Réponse : b (combinaison linéaire puis activation).

2. Quelle fonction d'activation donne une interprétation probabiliste en classification binaire ?

- (a) Seuil (Heaviside)
- (b) Sigmoidé
- (c) Tanh
- (d) ReLU

Réponse : b (sigmoidé, $\hat{y} \sim P(1|\mathbf{x})$).

3. Quelle activation est utilisée pour la classification multiclasse ?

- (a) Sigmoidé
- (b) Tanh
- (c) Softmax
- (d) Seuil

Réponse : c (softmax).

4. Le perceptron monocouche équivaut à :

- (a) La régression linéaire
- (b) La régression logistique
- (c) L'analyse discriminante
- (d) Les SVMs

Réponse : b (régression logistique).

5. Quel problème illustre la limite de la classification linéaire ?

- (a) Le XOR
- (b) Le AND
- (c) Le OR
- (d) Le clustering

Réponse : a (XOR, non séparable linéairement).

6. Comment s'appelle la technique de calcul du gradient dans un MLP ?

- (a) Gradient boosting
- (b) Rétropropagation (backpropagation)
- (c) Descente de gradient
- (d) Forward propagation

Réponse : b (rétropropagation du gradient).

7. Quelle approximation du gradient est utilisée pour les grands jeux de données ?

- (a) Gradient complet
- (b) SGD (minibatch)
- (c) Newton-Raphson
- (d) BFGS

Réponse : b (SGD minibatch).

8. Que pénalise la régularisation L_2 (weight decay) ?

- (a) La somme des poids en valeur absolue
- (b) Le carré de la norme des poids
- (c) Le nombre de couches
- (d) Le taux d'apprentissage

Réponse : b (carré de la norme $\|\mathbf{W}\|^2$).

9. Quel est le rôle d'un ensemble de validation ?

- (a) Apprendre les paramètres
- (b) Régler les hyper-paramètres
- (c) Calculer la perte finale
- (d) Initialiser les poids

Réponse : b (régler les hyper-paramètres).

10. Qui a proposé le neurone formel en 1943 ?

- (a) Rosenblatt
- (b) McCulloch & Pitts
- (c) LeCun
- (d) Hinton

Réponse : b (McCulloch & Pitts, 1943).

Références

- [1] W. S. McCulloch, W. Pitts. *A logical calculus of the ideas immanent in nervous activity*. Bull. Math. Biophysics, 5:115–133, 1943.
- [2] F. Rosenblatt. *The perceptron: A probabilistic model for information storage and organization in the brain*. Psychological Review, 65(6):386–408, 1958.
- [3] P. Werbos. *Beyond regression: New tools for prediction and analysis in the behavioral sciences*. Ph.D. thesis, Harvard University, 1974.
- [4] G. Cybenko. *Approximation by superpositions of a sigmoidal function*. Math. Control Signals Systems, 2(4):303–314, 1989.
- [5] Y. LeCun et al. *Backpropagation applied to handwritten zip code recognition*. Neural Computation, 1(4):541–551, 1989.